Contents

RAG with LangChain

Build a RAG help desk for Zomato: turn its policy PDFs, wiki pages and FAQs into a question-answering system, one LangChain component at a time.

`Zomato help desk` `LangChain` `Qdrant` `OpenRouter`

---

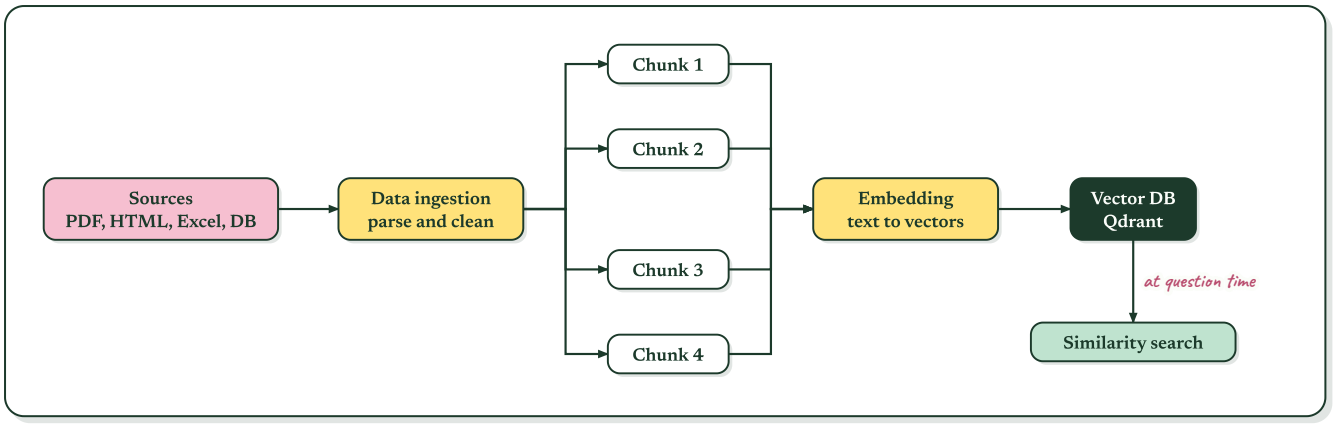
part one

What LangChain is, and what it is not

The big picture

RAG works in two steps: prepare your documents once, then retrieve the right pieces for every question and hand them to an LLM. This guide builds that system for real, for **Zomato**, the online food ordering platform. We will give Zomato's internal help desk an assistant that answers from the company's own knowledge, which is scattered across policy PDFs, scanned contracts, Markdown wiki pages and spreadsheets. We use LangChain for the plumbing and Qdrant as the vector database.

Start with the whole picture. Every source file is parsed, cut into chunks, turned into vectors and stored. Questions later search those vectors by similarity.



Now give each stage its LangChain name. Each of these components is covered below, in this order.

Indexing Document loader → Text splitter → Embeddings → QdrantVectorStore

Querying Retriever → Prompt template → Chat model → Output parser → Answer

tip

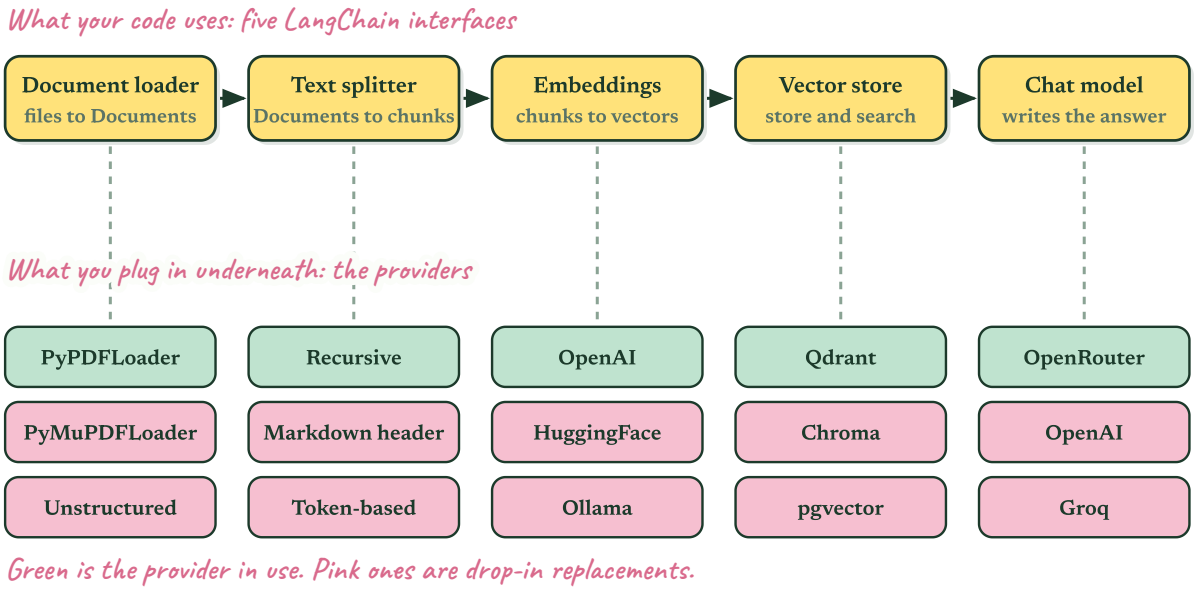
Keep one rule in mind all the way through: **a bad answer is almost always a retrieval problem, not a model problem.** LangChain makes every stage visible, so you can inspect each one.

Go deeper LangChain: build a RAG app LangChain: retrieval

What is LangChain?

LangChain is an open-source framework that gives every part of an LLM application a **standard interface**. A PDF parser, an embedding API, a vector database and a chat model all come from different vendors with different SDKs. LangChain wraps each one in the same few shapes, so you can connect them like building blocks and swap any block without

rewriting the rest. Read the diagram in two layers: the top row is what your code works with, and the stacks underneath are the providers that do the real work.



LangChain is the wrapper in the middle. Your code talks to the top row, so changing a provider underneath changes one line, not your pipeline.

Copy

```
# the pipeline code stays the same ...
chunks = splitter.split_documents(docs)
store = QdrantVectorStore.from_documents(chunks, embeddings, url=URL)

# ... only the provider line changes
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")          # ho
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")      # lo
```

What it gives you

- One interface per job, many providers behind it
- Ready-made loaders, splitters and vector-store wrappers
- A way to compose steps into a pipeline with `|`

What it is not

- It is not RAG. RAG is the design, LangChain is a toolbox.
- It is not a database. Qdrant stores the vectors.
- It is not a model. OpenAI and OpenRouter do the AI.

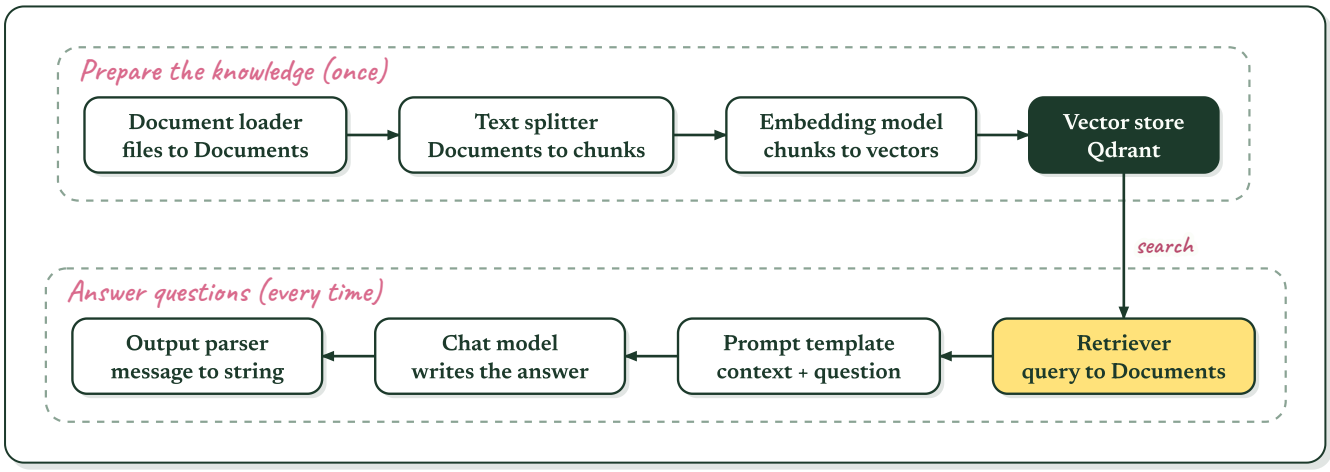
- Streaming, batching and tracing for free

LangChain is the orchestration layer. Qdrant stores, OpenAI embeds, OpenRouter generates.

Once you see these four roles clearly, every line of code below has an obvious home.

The components map

Eight components cover almost everything a RAG application needs. The top row prepares your knowledge once. The bottom row runs for every question, reading from the same vector store.



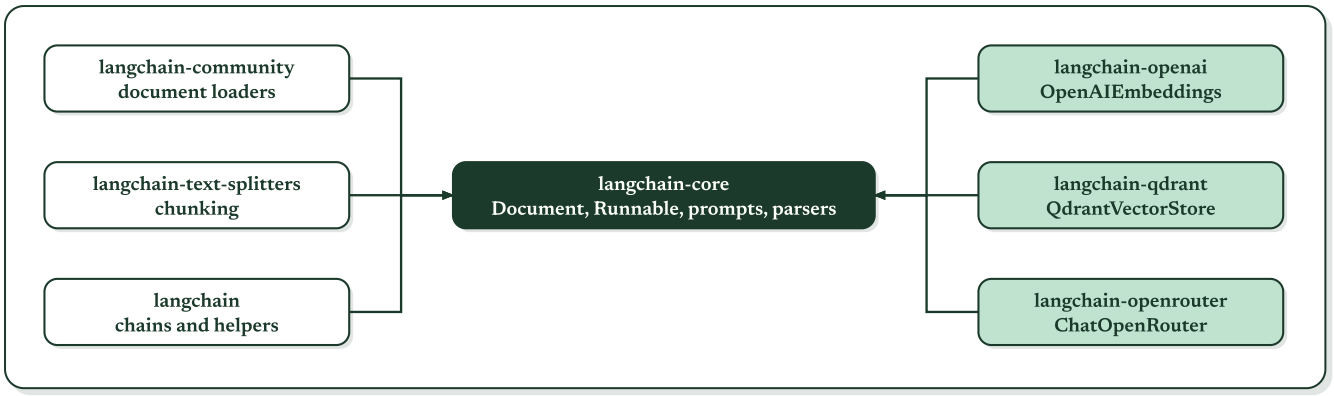
Component	Job	Example
Document loader	Read a source and return `Document` objects	`PyMuPDFLoader` , `TextLoader` , `CSVLoader`
Text splitter	Cut Documents into retrievable chunks	`RecursiveCharacterTextSplitter` , `MarkdownHeaderTextSplitter`

Component	Job	Example
Embedding model	Turn text into vectors	`OpenAIEmbeddings`
Vector store	Store vectors and search by similarity	`QdrantVectorStore`
Retriever	A simple "query in, Documents out" interface	`vector_store.as_retriever()`
Prompt template	Fill a reusable prompt with context and question	`ChatPromptTemplate`
Chat model	Generate the answer	`ChatOpenRouter`
Output parser	Turn the model message into a plain string	`StrOutputParser`

The glue between them is the **Runnable** interface. Every component above can be called with `.invoke()`, and any two can be chained with the pipe operator `|`. That is called LCEL, the LangChain Expression Language, and it comes into play when we assemble the full chain.

How the packages fit together

LangChain is split into small packages so that you install only what you use. The mental model is a hub and spokes: `langchain-core` defines the shared interfaces, and every other package plugs into it.



This is why imports come from different places. `Document` and prompts live in `langchain_core`. Loaders live in `langchain_community`. A provider such as OpenAI or Qdrant ships its own small package. When an import fails, the fix is almost always "install the matching package".

> *Go deeper* LangChain integrations catalogue All document loaders

---

Project setup

We will build a small help-desk assistant for Zomato. Assume the company's documents already sit in a `knowledge` folder, grouped by where they came from. The mix is deliberate: it is what a real company folder looks like, and each kind of file needs a different tool.

```
helpdesk-rag/
├── .env                              API keys, never commit this file
├── settings.py                       one place for every setting
├── knowledge/
│   ├── policies/
│   │   ├── employee_handbook.pdf     text PDF, 8 pages
│   │   └── expense_policy.pdf        text PDF with rate tables
│   ├── scans/
│   │   └── vendor_contract.pdf       scanned pages, no selectable text
│   ├── wiki/
│   │   ├── onboarding.md             Markdown with headings
│   │   └── payments_runbook.md       Markdown runbook
│   ├── faq/
```

```
│   │   └── support_faq.csv          question, answer, product, category
│   └── catalog/
│       └── plans.json               pricing plans and features
├── kb/                              our small library
│   ├── loaders.py                   files to Documents
│   ├── chunkers.py                  Documents to chunks
│   └── store.py                     Qdrant vector store
└── explore.ipynb                    where we experiment first
```

Install the packages

```
pip install -U langchain langchain-core langchain-community langchain-openai
    langchain-openrouter langchain-qdrant langchain-text-splitters \
    qdrant-client python-dotenv pypdf pymupdf
```

Copy

> *tip*
> In a Jupyter notebook use `%pip install -U ...` and restart the kernel afterwards if older LangChain packages were already loaded. Extra loaders and splitters need their own installs, noted where they appear.

The stack and the keys

Layer	Technology	Package
Embeddings	OpenAI `text-embedding-3-small`	`langchain-openai`
Vector database	Qdrant	`langchain-qdrant`
Chat model	Any model through OpenRouter	`langchain-openrouter`
Chunking	Several splitters, compared below	`langchain-text-splitters`
Loading	Several loaders, compared below	`langchain-community`

The embedding model and the chat model do **not** have to come from the same provider. Here OpenAI creates the vectors and OpenRouter writes the answers, so you need two keys in a `.env` file next to your code:

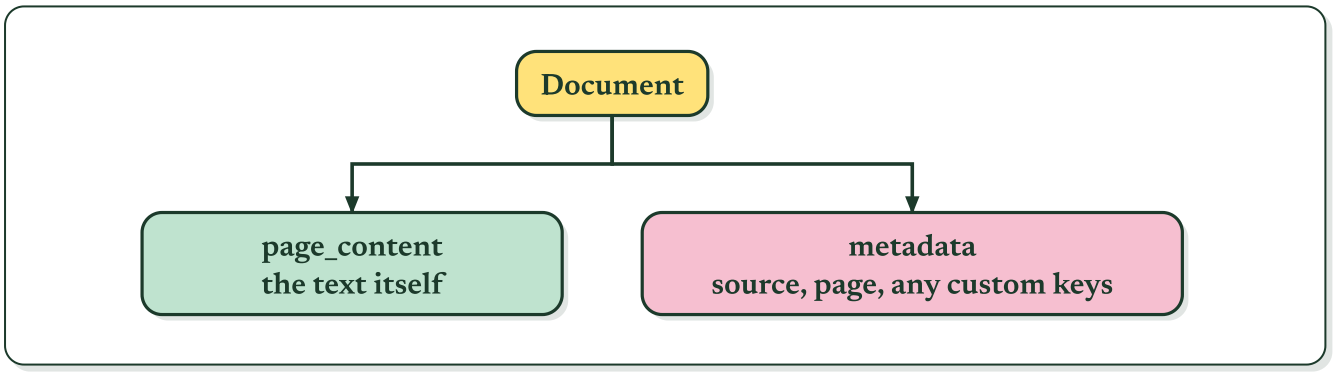
---

part two

From files to Documents

The Document object

Before loading anything, meet the one object that travels through the entire pipeline. A LangChain `Document` is just two fields: the text, and a dictionary describing where the text came from.




```
from langchain_core.documents import Document

doc = Document(
    page_content=(
        "Full-time employees receive 24 days of paid annual leave per calenda
        "Up to 5 unused days may be carried over into the next year."
    ),
    metadata={"filename": "employee_handbook.pdf", "page": 1, "area": "polic
)

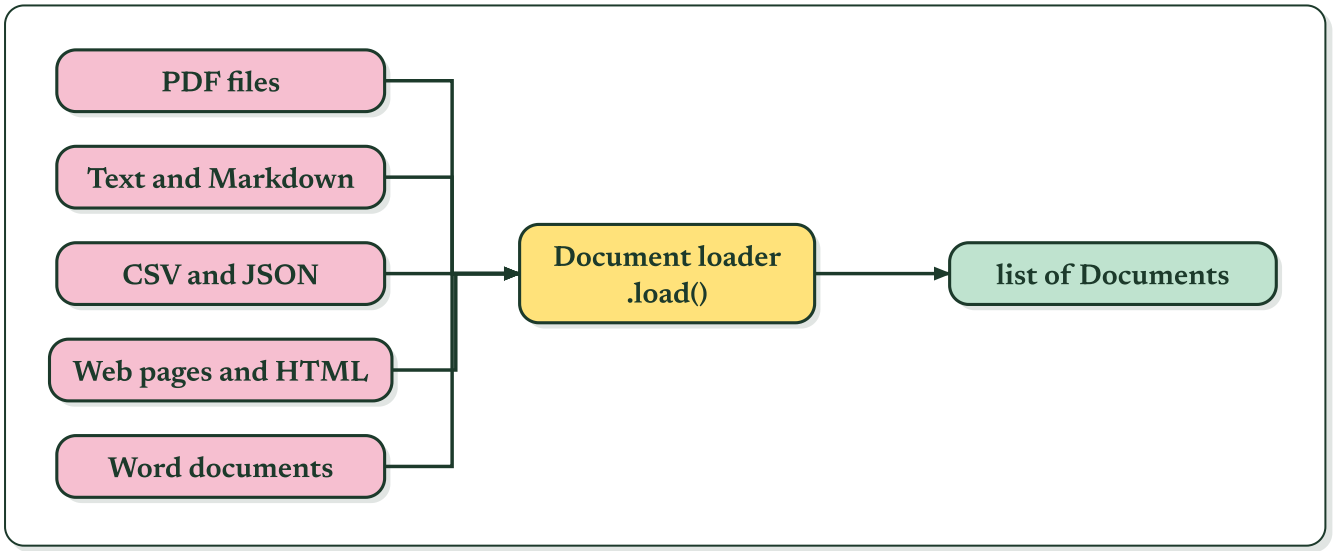
print(doc.page_content)
print(doc.metadata)
```

Why bother with metadata? Because the vector store needs more than text. Metadata powers **source citations** ("see page 2 of the handbook"), **filtering** (search only the policies), **tenant isolation**, access control and debugging. Every later step, loading, splitting and storing, carries it along automatically.

> *Go deeper* Document API reference

Document loaders

A loader's only job is to read an external source and return a list of `Document` objects. The source can be a PDF, a text file, a CSV, a web page or a database. The shape of the output is always the same. What differs between loaders is **how well they read a particular kind of file**, and that is where your choice matters.



The next few sections walk through the main families, then compare the options that overlap, and finish with a decision guide. Browse the full loader catalogue when you meet a format not covered here. There are hundreds, and most live in `langchain-community`.

Loading text and Markdown

The simplest case is a plain file. `TextLoader` reads the whole file into one Document and records the path as `source`.

```
from langchain_community.document_loaders import TextLoader

notes = TextLoader("knowledge/wiki/onboarding.md", encoding="utf-8").load()

print(len(notes))                    # 1 file, so 1 Document
print(notes[0].metadata)             # {'source': 'knowledge/wiki/onboarding.m
print(notes[0].page_content[:200])
```

Copy

> *tip*
> Always pass `encoding="utf-8"`. On Windows the default encoding can corrupt accented characters and symbols.

For Markdown there is a second option, and the difference is worth knowing:

Loader	What you get	Choose it when
`TextLoader`	The raw text, `#` headings included, as one Document	You will split on those headings yourself (recommended, see the splitting part)
`UnstructuredMarkdownLoader`	Cleaned text. In `mode="elements"` it returns titles, paragraphs and lists as separate Documents	You want element-level structure and accept the heavier `unstructured` dependency

For most Markdown, `TextLoader` plus a header-aware splitter is lighter, faster and keeps your headings exactly as written.

Loading a PDF

PDFs are what most real knowledge bases look like: handbooks, policies, contracts, manuals. A PDF loader returns **one Document per page** and fills metadata with the page number and whatever the file declares about itself. We start with `PyPDFLoader`, the most common entry point.

```
from langchain_community.document_loaders import PyPDFLoader

pages = PyPDFLoader("knowledge/policies/employee_handbook.pdf").load()

print(f"Pages loaded: {len(pages)}")
print(pages[1].page_content[:240])
print(pages[1].metadata)
```

Copy

```
Pages loaded: 8
4.1 Annual leave
Full-time employees receive 24 days of paid annual leave per calendar year.
Up to 5 unused days may be carried over into the next year ...

{'producer': 'Microsoft Word', 'creator': 'Microsoft Word', 'creationdate':
 'source': 'knowledge/policies/employee_handbook.pdf', 'total_pages': 8, 'pa
```

Notice two page fields. `page` counts from 0 (the index in the file), while `page_label` is the number printed on the page. Use the label when you show citations to humans. Output above is illustrative of what you will see on your own files.

> *Go deeper* PyPDFLoader reference

PDF loaders compared

LangChain ships several loaders for the same job, because PDFs are the messiest format in business. They differ in speed, in how faithfully they preserve reading order, and in what extra they give you. Three cover almost every text-based PDF you will meet:

Loader	Install	Strengths	Weaknesses
`PyPDFLoader`	`pypdf`	Pure Python, trivial to install, page labels in metadata	Can scramble columns and tables. Mid-range speed.
`PyMuPDFLoader`	`pymupdf`	Very fast, good reading order, rich metadata	Native dependency. Check the licence (AGPL) for commercial use.
`PDFPlumberLoader`	`pdfplumber`	Precise character positions, helpful when you later extract tables with the pdfplumber API	Slower than the others

Do not pick by reputation. Run each candidate on your own hardest PDF and read the result. This small harness takes a minute and often settles the argument:

```
import time
from langchain_community.document_loaders import PDFPlumberLoader, PyMuPDFLo

path = "knowledge/policies/expense_policy.pdf"      # a PDF that contains a
candidates = {"pypdf": PyPDFLoader, "pymupdf": PyMuPDFLoader, "pdfplumber": P

for name, loader_cls in candidates.items():
    started = time.perf_counter()
    pages = loader_cls(path).load()
    seconds = time.perf_counter() - started
    chars = sum(len(p.page_content) for p in pages)
```

```
    print(f"{name:>10}: {len(pages)} pages, {chars:,} characters, {seconds:.2
    print("            sample:", pages[2].page_content[:90].replace("\n", "
```

note

What to look for in the sample: are table rows still readable, or did the amounts drift away from their labels? Is the left column read before the right column? Choose the loader whose output you would be happy to hand to an LLM.

tip

Rule of thumb. Start with `PyMuPDFLoader` for speed and quality, fall back to `PyPDFLoader` when you need zero native dependencies, and try `PDFPlumberLoader` when tables matter. Scanned files need OCR, which the next section covers.

Scanned PDFs need OCR

Open a PDF and try to select a sentence with your mouse. If you cannot, the pages are **pictures of text.** A scanned contract is the classic example. Every loader above would return empty or garbage pages, because there is no text layer to read. You need OCR, optical character recognition, which looks at the pixels and guesses the letters.

Scanned PDF (images) → OCR engine → Recognised text → Document objects

How OCR works technically

When you set `strategy="ocr_only"`, the `unstructured` library ignores text layers and treats pages as images. It orchestrates three steps:

- **Rendering:** `poppler` or `pdf2image` converts PDF pages into image buffers.
- **Recognition:** The **Tesseract OCR** engine (maintained by Google) scans pixels to identify characters and bounding boxes.
- **Partitioning:** The library uses those boxes to group text into *elements* like Titles, NarrativeText, or Tables.

Note: System tools (`tesseract` and `poppler`) must be preinstalled on your OS for this to work.

```
pip install "unstructured[pdf]"
# OCR also needs system tools: tesseract and poppler (for example: brew insta
```

Copy

```
from langchain_community.document_loaders import UnstructuredPDFLoader

scan = UnstructuredPDFLoader(
    "knowledge/scans/vendor_contract.pdf",
    mode="elements",            # titles, paragraphs and tables as separate
    strategy="ocr_only",        # force OCR instead of reading a text layer
).load()

print(len(scan), "elements")
print(scan[0].metadata.get("category"), "|", scan[0].page_content[:80])
```

Copy

watch out

OCR is slower and makes mistakes (an "O" read as a zero, a missing comma). For contracts and financial documents, always spot-check the recognised text, and keep the original file as the source of truth for any legal question.

When PDFs are truly hard, with merged table cells, multi-column layouts or handwriting, teams often move to a managed parser such as LlamaParse, Azure Document Intelligence or Amazon Textract. They cost money but handle layouts that open-source tools cannot.

CSV, JSON, Word and web loaders

Not every source is a PDF. Each format below has a dedicated loader, and the right one saves you from hand-written parsing code.

CSV: one row, one Document

The support team keeps frequently asked questions in a spreadsheet. `CSVLoader` turns every row into a Document, and `metadata_columns` lifts chosen columns into metadata so you can filter on them.

```
from langchain_community.document_loaders import CSVLoader

faq = CSVLoader(
    "knowledge/faq/support_faq.csv",
    encoding="utf-8",
    metadata_columns=["product", "category"],
).load()

print(len(faq), "rows")
print(faq[0].page_content)
print(faq[0].metadata)
```

Copy

```
63 rows
question: Which plan includes single sign-on?
answer: SSO is available on the Business and Enterprise plans.
{'source': 'knowledge/faq/support_faq.csv', 'row': 0, 'product': 'Zomato for
```

JSON: choose what becomes text

JSON has no natural text, so you tell the loader what to extract using a `jq` expression. Think of `jq` as a query language for JSON: you describe a *path* through the document and the loader turns each thing that path lands on into one Document.

A few rules cover almost every case you will need:

Expression	What it does
`.`	The whole document as one Document.
`.product`	Follows one key. `.billing.cycle` walks into nested objects.
`.plans`	The whole array as a single Document.

Expression	What it does
`.plans[]`	Every item in the array becomes its **own** Document. This is the one you want most of the time.
`.plans[] | .name`	Pipes the result onward, so each Document keeps only the `name` value.
`.plans[] | {name, sso}`	Builds a smaller object per item, keeping just the fields you choose.

Compare each result below on the same file. `text_content=False` keeps the extracted JSON as text; leave it out to flatten the values into a plain string instead.

Copy

```
# pip install jq
from langchain_community.document_loaders import JSONLoader

# ".plans[]" -> one Document per plan (3 plans = 3 Documents)
plans = JSONLoader(
    file_path="knowledge/catalog/plans.json",
    jq_schema=".plans[]",
    text_content=False,          # keep each plan as JSON text
).load()

print(len(plans), "documents")   # 3
print(plans[0].page_content)     # the Starter plan, as JSON

# ".plans[] | {name, sso}" -> each Document keeps only those two fields
compact = JSONLoader(
    file_path="knowledge/catalog/plans.json",
    jq_schema=".plans[] | {name, sso}",
    text_content=False,
).load()

# ".billing" -> a single Document holding just the billing block
billing = JSONLoader(
    file_path="knowledge/catalog/plans.json",
    jq_schema=".billing",
    text_content=False,
).load()
```

```
# metadata_func: lift a field into metadata so you can filter on it later
def plan_meta(record: dict, metadata: dict) -> dict:
    metadata["plan"] = record.get("name")
    return metadata

per_plan = JSONLoader(
    file_path="knowledge/catalog/plans.json",
    jq_schema=".plans[]",
    text_content=False,
    metadata_func=plan_meta,
).load()
print(per_plan[1].metadata["plan"])   # Business
```

tip

Keep one thing in mind while writing a jq path. A trailing `[]` explodes an array into many Documents; a named key keeps things together. When your loader returns "too few" or "too many" Documents, the fix is almost always the shape of that path.

watch out

A missing path does not raise an error, it silently yields nothing. If your JSON loader returns zero Documents, print the file and check the exact key names before assuming the loader is broken.

Word, HTML and live web pages

Copy

```
from langchain_community.document_loaders import BSHTMLLoader, Docx2txtLoader

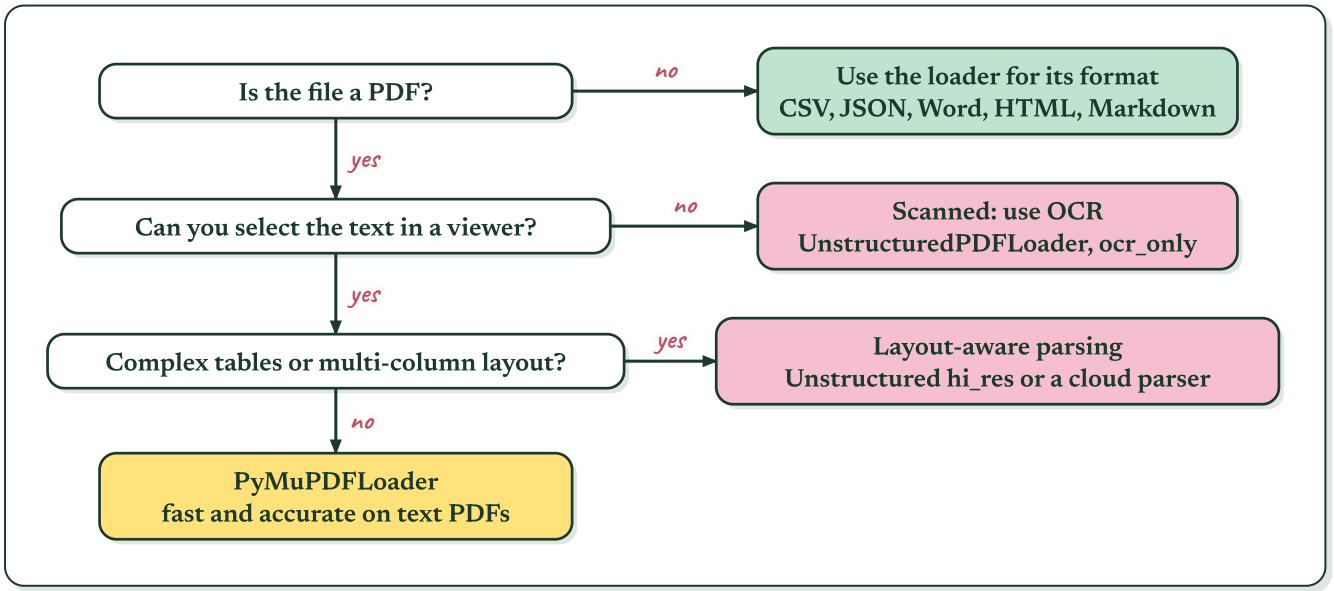
memo = Docx2txtLoader("knowledge/policies/travel_memo.docx").load()
page = BSHTMLLoader("knowledge/wiki/holiday_calendar.html").load()
site = WebBaseLoader(["https://example.com/help/shipping-rates"]).load()
```

Format	Loader	Gives you	Watch out for
CSV	`CSVLoader`	One Document per row	Very long rows. Pick the columns that matter.

Format	Loader	Gives you	Watch out for
JSON	`JSONLoader`	Documents chosen by a jq expression	Needs `jq` . You must decide what counts as text.
Word	`Docx2txtLoader`	The whole document as one Document	Tables flatten to plain text. Use the Unstructured Word loader for structure.
HTML file	`BSHTMLLoader`	Visible text plus the page title	Menus and footers leak into the text
Live web page	`WebBaseLoader`	Text from one or more URLs	Respect robots.txt and rate limits. Pages change over time.

Which loader should you use?

Walk down this tree for any new document. The first question that matters is whether a human can select the text.



Real-world cheat sheet

Scenario	Choose	Why
Employee handbook exported from Word	`PyMuPDFLoader`	A text PDF with simple layout. Speed and fidelity are both good.
Signed vendor contract scanned on a copier	`UnstructuredPDFLoader` with OCR	No text layer exists, so only OCR can read it.
Expense policy with rate tables	Try `PDFPlumberLoader`, then `Unstructured`	Tables need position-aware extraction or a layout model.
Support FAQ spreadsheet	`CSVLoader`	Each row is already one question and one answer.
Engineering runbook in Markdown	`TextLoader`	Keep the headings, then split on them.
A minimal environment with no native packages	`PyPDFLoader`	Pure Python, installs everywhere.

Loading a whole folder

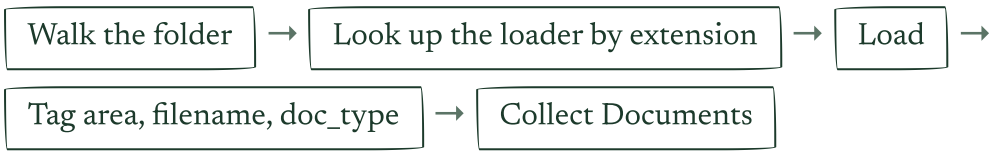
Real projects have many files, not one. LangChain offers `DirectoryLoader` for the quick version: it scans a folder and applies one loader class to every file matching a pattern.

```
from langchain_community.document_loaders import DirectoryLoader, PyMuPDFLoad

loader = DirectoryLoader(
    "knowledge/policies",
    glob="**/*.pdf",
    loader_cls=PyMuPDFLoader,
    show_progress=True,
```

```
)
docs = loader.load()
```

That works until your folder mixes formats, and ours does. One loader class cannot read PDFs, CSVs and Markdown at once. A common design is a small **registry**: a dictionary from file extension to the loader that should read it. It stays readable, it is easy to extend, and one corrupt file cannot crash the whole run.



kb/loaders.py

```
"""Turn the files in the knowledge folder into LangChain Documents."""
from pathlib import Path

from langchain_community.document_loaders import (
    CSVLoader, PyMuPDFLoader, TextLoader, UnstructuredPDFLoader,
)
from langchain_core.documents import Document

SCAN_FOLDERS = {"scans"}      # PDFs kept here are images, so they need OCR


def _pdf(path: Path):
    if path.parent.name in SCAN_FOLDERS:
        return UnstructuredPDFLoader(str(path), mode="elements", strategy="oc
    return PyMuPDFLoader(str(path))


LOADER_FOR = {
    ".pdf": _pdf,
    ".md": lambda p: TextLoader(str(p), encoding="utf-8"),
    ".txt": lambda p: TextLoader(str(p), encoding="utf-8"),
    ".csv": lambda p: CSVLoader(str(p), encoding="utf-8", metadata_columns=['
}


def load_file(path: Path) -> list[Document]:
    docs = LOADER_FOR[path.suffix.lower()](path).load()
```

```
        for doc in docs:
            doc.metadata.update(
                filename=path.name,
                area=path.parent.name,                # policies, wiki, faq, sc
                doc_type=path.suffix.lstrip(".").lower(),
            )
        return docs


    def load_corpus(root: Path) -> list[Document]:
        docs, failed = [], []
        for path in sorted(root.rglob("*")):
            if path.suffix.lower() not in LOADER_FOR:
                continue
            try:
                docs.extend(load_file(path))
            except Exception as exc:                  # keep going, report at t
                failed.append(f"{path.name}: {exc}")
        print(f"Loaded {len(docs)} documents from {root}")
        for line in failed:
            print("  skipped", line)
        return docs
```

Copy

```
from settings import KNOWLEDGE_DIR
from kb.loaders import load_corpus

docs = load_corpus(KNOWLEDGE_DIR)
```

```
Loaded 97 documents from knowledge
  skipped vendor_contract.pdf: tesseract is not installed
```

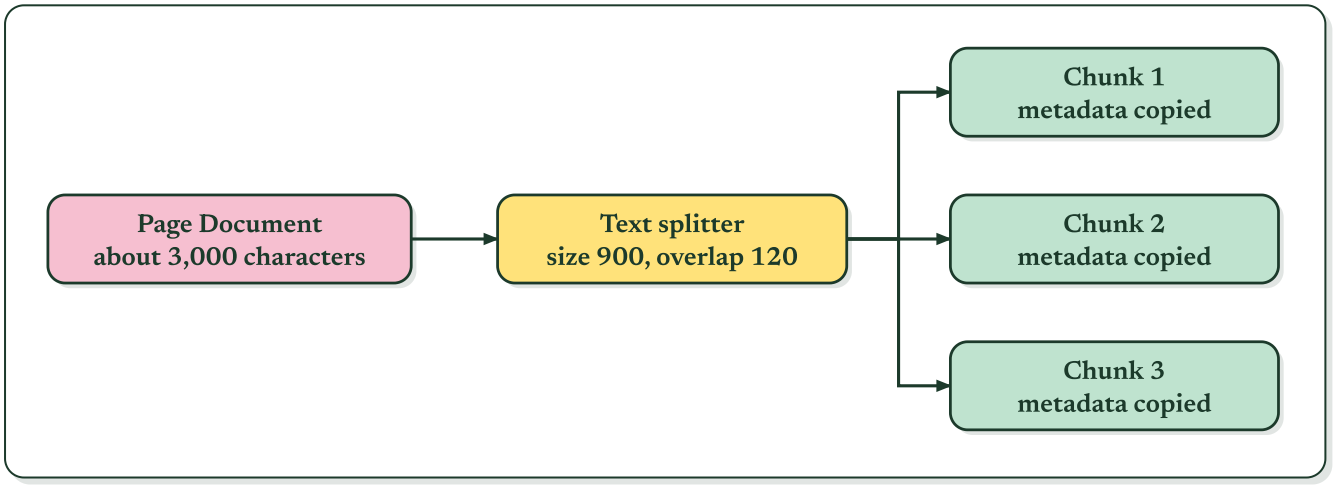
The output above is an example. The point of the design is that the skipped file is *reported*, not silently ignored and not fatal. Install the missing tool and run again.

part three

From Documents to chunks

Splitting into chunks

A page is still too big and too mixed. A handbook page about annual leave may also contain the first half of the sick-leave rules. We split documents into chunks small enough to be about one thing, because a retrieved chunk is exactly what the LLM will read. LangChain's most common splitter is `RecursiveCharacterTextSplitter` : it tries paragraph breaks first, then line breaks, then spaces, so chunks tend to end at natural boundaries.



Copy

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

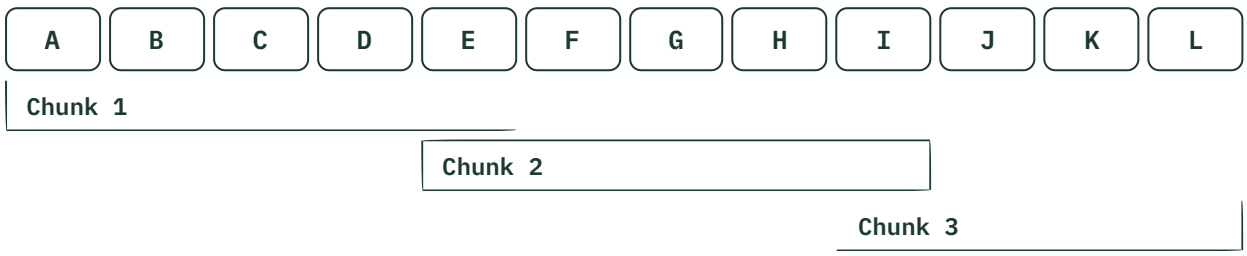
splitter = RecursiveCharacterTextSplitter(
    chunk_size=900,          # maximum characters per chunk
    chunk_overlap=120,       # characters shared between neighbouring chunks
    add_start_index=True,    # remembers where in the page each chunk began
)

chunks = splitter.split_documents(docs)

print(f"Split {len(docs)} documents into {len(chunks)} chunks")
print(chunks[0].page_content[:200])
print(chunks[0].metadata)
```

Use `split_documents()`, not `split_text()`. The first keeps the Document abstraction, so every chunk inherits its page's metadata automatically. With `split_text()` you get bare strings and lose the source, and then you cannot cite or filter.

The overlap repeats the end of one chunk at the start of the next, so an idea that straddles a boundary survives. With size 5 and overlap 1:



size 5, overlap 1, 3 chunks

> *Go deeper* RecursiveCharacterTextSplitter reference Pinecone: chunking strategies

Splitters compared

The recursive splitter is a great default, but it is one of several. The splitters below solve different problems. The main differences are **what they cut on** and **how they measure size**.

Splitter	Cuts on	Size measured in
`CharacterTextSplitter`	One separator you choose, such as a blank line	Characters

Splitter	Cuts on	Size measured in
`RecursiveCharacterTextSplitter`	Paragraph, then line, then space, as needed	Characters
Token-based (`from_tiktoken_encoder` , `TokenTextSplitter`)	Token boundaries	Tokens
`MarkdownHeaderTextSplitter`	Markdown headings	Sections
`HTMLHeaderTextSplitter`	HTML heading tags	Sections
`RecursiveCharacterTextSplitter.from_language()`	Classes, functions and blocks of a language	Characters
`RecursiveJsonSplitter`	JSON structure	Serialised size

Splitter	Cuts on	Size measured in
`SemanticChunker`	Points where the meaning shifts	Sentences

Let us see them work, starting with the three general-purpose ones on the same text.

```
from langchain_text_splitters import CharacterTextSplitter, RecursiveCharacte

text = open("knowledge/wiki/onboarding.md", encoding="utf-8").read()

# 1. cuts only at blank lines, then merges paragraphs up to the size limit
by_char = CharacterTextSplitter(separator="\n\n", chunk_size=500, chunk_overl

# 2. tries blank lines, then newlines, then spaces, so oversized paragraphs
by_recursive = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=5

# 3. sizes in tokens, matching what the embedding model actually counts
by_tokens = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base", chunk_size=120, chunk_overlap=15
)

for name, splitter in [("character", by_char), ("recursive", by_recursive),
    pieces = splitter.split_text(text)
    longest = max(len(p) for p in pieces)
    print(f"{name:>10}: {len(pieces):>3} chunks, longest = {longest} characte
```

Copy

watch out

`CharacterTextSplitter` only cuts at its one separator. If a single paragraph is longer than `chunk_size`, it keeps that paragraph whole and the chunk silently exceeds your limit. That is the main reason the recursive splitter is the safer default.

tokens

What is a token?

A **token** is the small piece of text a language model actually reads. It can be a whole word, part of a word, a single character, a punctuation mark, or a space glued to the front of a word. A **tokenizer** cuts your text into pieces taken from a fixed vocabulary of tens of thousands of entries, and every piece has an integer ID. The model never sees letters. It sees a list of numbers.

What do tokens look like?

Here is one sentence cut by three real encodings. Each colour is one token, and `·` marks a leading space, which belongs to the token.

"Zomato delivers unbelievably fast."

r50k_base	`Z` `om` `ato` `·delivers` `·unbelievably` `·fast` `.`	7 tokens
cl100k_base	`Z` `om` `ato` `·delivers` `·unbelie` `vably` `·fast` `.`	8 tokens
o200k_base	`Z` `om` `ato` `·delivers` `·unbelievably` `·fast` `.`	7 tokens

Underneath, the model works with IDs. These are the same eight tokens from `cl100k_base`:

IDs	`Z`	`om`	`ato`	`·delivers`	`·unbelie`	`vably`	`·fast`	`.`
	57	316	4428	28421	40037	89234	5043	13

"Annual leave: 24 days, carry over 5."

r50k_base	`Ann` `ual` `·leave` `:` `·24` `·days` `,` `·carry` `·over` `·5` `.`	11 tokens
cl100k_base	`Annual` `·leave` `:` `·` `24` `·days` `,` `·carry` `·over` `·` `5` `.`	12 tokens
o200k_base	`Annual` `·leave` `:` `·` `24` `·days` `,` `·carry` `·over` `·` `5` `.`	12 tokens

Why do different encodings give different tokens?

Each encoding learned its own vocabulary from its own training text. The older `r50k_base` has about 50,000 entries, `cl100k_base` about 100,000 and `o200k_base` about 200,000. Look at what changed above: the same word, "unbelievably", is one token in two encodings and two tokens in the middle one, and the older encoding keeps " 24" as one piece while the newer ones split the space from the digits. A rare brand name like "Zomato" is cut into three pieces by all of them, because none of the vocabularies holds it as a whole. As a rough guide, one token is about four characters, or three quarters of an English word.

Why use tokens at all?

- **No unknown words.** Any text, including new names, typos and code, can be assembled from known pieces, so the model never hits a word it cannot represent.
- **A sensible middle ground.** Single characters would make every text very long and slow to process. Whole words would need a vocabulary of millions. Pieces of words keep common words short and rare words possible.
- **Numbers a model can use.** Each token ID selects a learned vector from the embedding table, and that is where all the model's maths starts.

How LLMs use tokens

An LLM reads your prompt as a list of token IDs and predicts the single most suitable *next* token. It appends that token, predicts the next one, and repeats until the answer is complete, so a reply is a stream of tokens converted back into text. Because everything is counted in tokens, so are the limits and the bill: the **context window** (how much the model can read at once), the maximum reply length, the price per request, and the input limit of an embedding model.

Why this matters for RAG

The retrieved chunks, your prompt and the answer all have to fit in one token budget. The same text can cost a different number of tokens on different models, and non-English text, numbers and code often cost more than you would guess. When a limit matters, count with the tokenizer that belongs to your model instead of estimating from characters.

```
import tiktoken            # pip install tiktoken

enc = tiktoken.get_encoding("cl100k_base")      # the encoding used by text-
ids = enc.encode("Zomato delivers unbelievably fast.")

print(ids)                                      # [57, 316, 4428, 28421, 40037, 
print([enc.decode([i]) for i in ids])           # ['Z', 'om', 'ato', ' delivers'
print(len(ids), "tokens")                       # 8 tokens
```

Copy

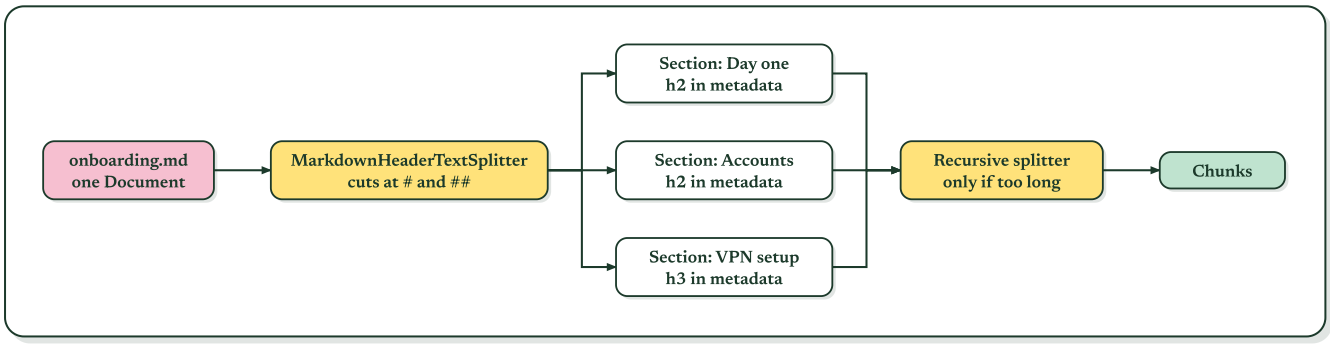
That is why the token-based splitter earlier exists: when you need a hard guarantee such as "never more than 300 tokens per chunk", size the chunks in tokens, not characters.

> *Go deeper* OpenAI Tokenizer: paste any text and see its tokens

---

Header-aware splitting for Markdown and HTML

Structured documents already tell you where the topics change. A heading like "Restarting the payments service" is a far better boundary than "after 900 characters". `MarkdownHeaderTextSplitter` cuts at headings and, as a bonus, stores the heading path in metadata, so a chunk knows *which section it belongs to*.




```
from langchain_text_splitters import MarkdownHeaderTextSplitter, RecursiveCha

header_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "h1"), ("##", "h2"), ("###", "h3")],
)

markdown = open("knowledge/wiki/onboarding.md", encoding="utf-8").read()
sections = header_splitter.split_text(markdown)

print(len(sections), "sections")
print(sections[2].metadata)
print(sections[2].page_content[:150])
```

Copy

```
9 sections
{'h1': 'Onboarding', 'h2': 'Accounts and access', 'h3': 'VPN setup'}
Install the company VPN client from the IT portal, then sign in with your wo
```

The best pattern is **two stages**: split on headings first, then run the recursive splitter on any section that is still too long. You get meaningful boundaries *and* a size cap.

```
size_cap = RecursiveCharacterTextSplitter(chunk_size=900, chunk_overlap=120,
chunks = size_cap.split_documents(sections)      # sections that already fit
```

Copy

HTML pages work the same way with `HTMLHeaderTextSplitter`, which cuts at `<h1>` to `<h3>` tags:

```
from langchain_text_splitters import HTMLHeaderTextSplitter

html_splitter = HTMLHeaderTextSplitter(headers_to_split_on=[("h1", "title"),
parts = html_splitter.split_text(open("knowledge/wiki/holiday_calendar.html"
```

Copy

> *tip*
> Header metadata is gold for retrieval and for citations. You can show "Onboarding > Accounts and access > VPN setup" next to an answer, and you can filter for `h2 == "Accounts and access"` later.

> *Go deeper* LangChain text splitter integrations

Splitting code and JSON

Prose rules do badly on code. A function cut in half is useless. LangChain's recursive splitter can be told which language it is reading, and then it prefers to cut between classes and functions.

```
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

source = open("knowledge/wiki/retry_helper.py", encoding="utf-8").read()

code_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=600,
    chunk_overlap=60,
)
code_chunks = code_splitter.create_documents([source])
print(len(code_chunks), "chunks, each ending near a function boundary")
```

Many languages are supported (Python, JavaScript, Java, Go, Markdown and more). JSON has its own splitter that keeps objects intact and only splits where a nested structure allows it.

```
import json
from langchain_text_splitters import RecursiveJsonSplitter

data = json.load(open("knowledge/catalog/plans.json", encoding="utf-8"))

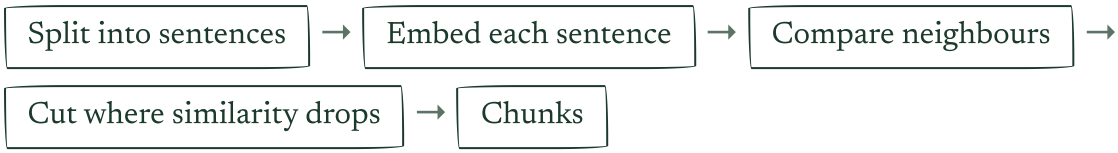
json_splitter = RecursiveJsonSplitter(max_chunk_size=400)
json_docs = json_splitter.create_documents(texts=[data])
print(len(json_docs), "JSON chunks")
```

> *note*
> Match the splitter to the *structure* of the file. A code splitter on a handbook, or a prose splitter on source code, works but throws away the clues the file was giving you.

Semantic chunking

Every splitter so far cuts by position or structure. **Semantic chunking** cuts by *meaning*. It embeds each sentence, measures how similar neighbouring sentences are, and starts a new

chunk where the similarity suddenly drops, which is where the topic changes.




```
pip install langchain-experimental
```

Copy

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

semantic = SemanticChunker(
    OpenAIEmbeddings(model="text-embedding-3-small"),
    breakpoint_threshold_type="percentile",     # cut at the biggest drops i
)

chunks = semantic.split_documents(docs[:5])
print(len(chunks), "semantic chunks")
```

Copy

Pros

- Chunks follow topics, not character counts
- Works on text with no headings

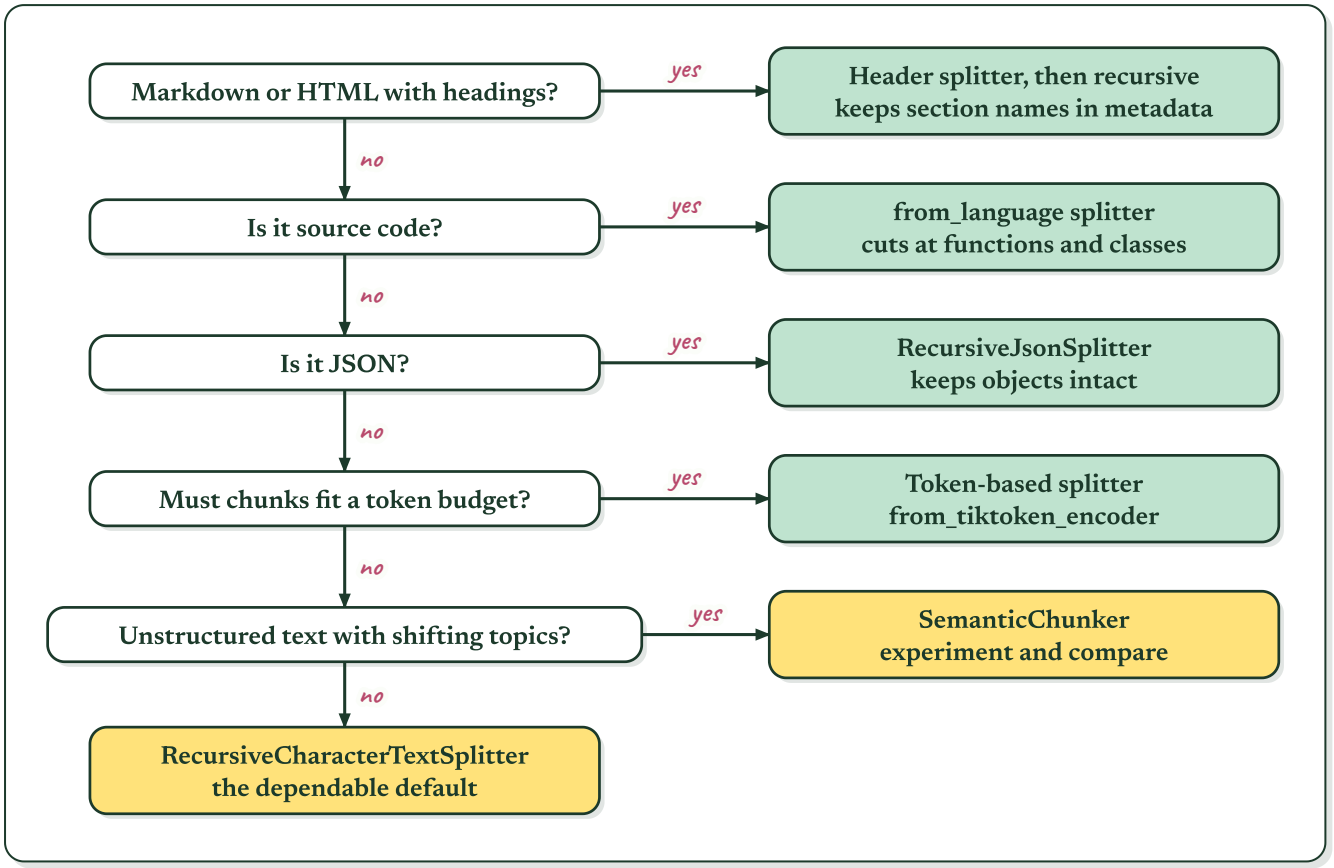
Cons

- Costs an embedding call for every sentence
- Chunk sizes are unpredictable
- Lives in the experimental package
- Often no better than headings plus recursive splitting

Treat it as an experiment to run when structure-based splitting gives poor answers, not as the default.

Which splitter should you use?

Start from the structure of what you are splitting, not from what is fashionable. This tree covers most real cases:



Real-world cheat sheet

Scenario	Choose	Why
Employee handbook PDF	Recursive, size 900	Prose with paragraphs. No headings survive the PDF.
Onboarding and runbook wiki pages	Markdown header splitter, then recursive	Each runbook step lives under its own heading.
Support FAQ rows	No splitting at all	One row is already one self-contained question and answer.
Product catalog JSON	`RecursiveJsonSplitter`	Keeps each plan's fields together.

Scenario	Choose	Why
Internal Python utilities	`from_language(Language.PYTHON)`	Whole functions stay together, so retrieved code is usable.
Strict embedding limit per chunk	Token-based	Guarantees the limit instead of estimating it.

> *Respect the structure the document already has.*
>
> Split on headings, functions and rows when they exist. Fall back to size-based splitting only for the leftover prose.

Choosing chunk size

Whichever splitter you pick, you still choose a size. There is no universal best, so measure. A cheap first experiment is to count how many chunks each setting produces, then read real chunks and ask whether each one reads like a self-contained idea.

```
for size, overlap in [(400, 40), (900, 120), (1500, 200)]:
    trial = RecursiveCharacterTextSplitter(chunk_size=size, chunk_overlap=ove
    print(size, overlap, "->", len(trial.split_documents(docs)), "chunks")
```

Copy

If you choose	You get	Typical symptom
Small chunks (about 300 to 500)	Precise matches, many vectors, more storage and embedding cost	Answers miss context because the idea was split in two

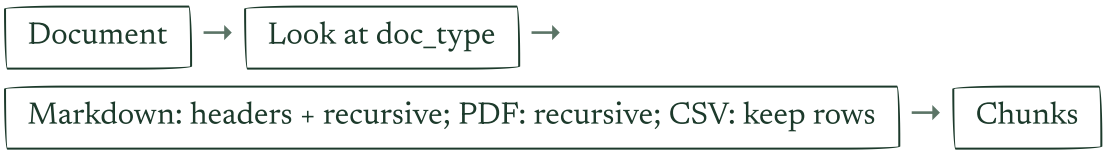
If you choose	You get	Typical symptom
Medium chunks (about 800 to 1200)	A good starting point for policies and manuals	Rarely a problem. Tune from here.
Large chunks (1500 and up)	More context per hit, fewer vectors	Noisy context, higher token cost, weaker precision

note

The retrieved chunks must also fit in the LLM's context window with the question and the prompt. With `k = 4` chunks of 900 characters you send roughly 3,600 characters of context, which any modern model handles comfortably.

One chunking module for every document type

Our folder has several kinds of content, and each deserves its own treatment. Instead of one splitter for everything, route each document to the right one by its type. This is the pattern real projects settle on.



kb/chunkers.py

```
"""Turn Documents into chunks, using the splitter that suits each document ty
from langchain_core.documents import Document
from langchain_text_splitters import MarkdownHeaderTextSplitter, RecursiveCha

_by_size = RecursiveCharacterTextSplitter(chunk_size=900, chunk_overlap=120,
_by_header = MarkdownHeaderTextSplitter(headers_to_split_on=[("#", "h1"), ("#
```

```
def _split_markdown(doc: Document) -> list[Document]:
    sections = _by_header.split_text(doc.page_content)
    for section in sections:
        section.metadata = {**doc.metadata, **section.metadata}   # keep fil
    return _by_size.split_documents(sections)


def chunk_corpus(docs: list[Document]) -> list[Document]:
    chunks: list[Document] = []
    for doc in docs:
        kind = doc.metadata["doc_type"]
        if kind == "md":
            chunks.extend(_split_markdown(doc))
        elif kind == "csv":
            chunks.append(doc)                        # a row is already one
        else:
            chunks.extend(_by_size.split_documents([doc]))
    print(f"{len(docs)} documents became {len(chunks)} chunks")
    return chunks
```

Metadata and stable chunk IDs

Last stop before storing. Everything you attach now becomes searchable later. Two additions pay off immediately in real projects: a **tenant or owner**, so you can isolate customers, and a **stable chunk ID**, so re-running ingestion never creates duplicates.

The simplest stable ID is a hash of the chunk's own text plus its file name. The same text in the same file always yields the same ID, so storing it twice just overwrites it.

```
import hashlib
import uuid


def chunk_id(chunk) -> str:
    digest = hashlib.sha1(chunk.page_content.encode("utf-8")).hexdigest()
    return str(uuid.uuid5(uuid.NAMESPACE_URL, f"{chunk.metadata['filename']}
```

```
for chunk in chunks:
    chunk.metadata["tenant_id"] = "zomato"

ids = [chunk_id(c) for c in chunks]
print(len(ids), len(set(ids)))       # equal, unless two chunks in one file
```

`uuid5` is a deterministic hash, so the same input always produces the same ID. That single property prevents a very common bug, as the re-ingestion section below shows.

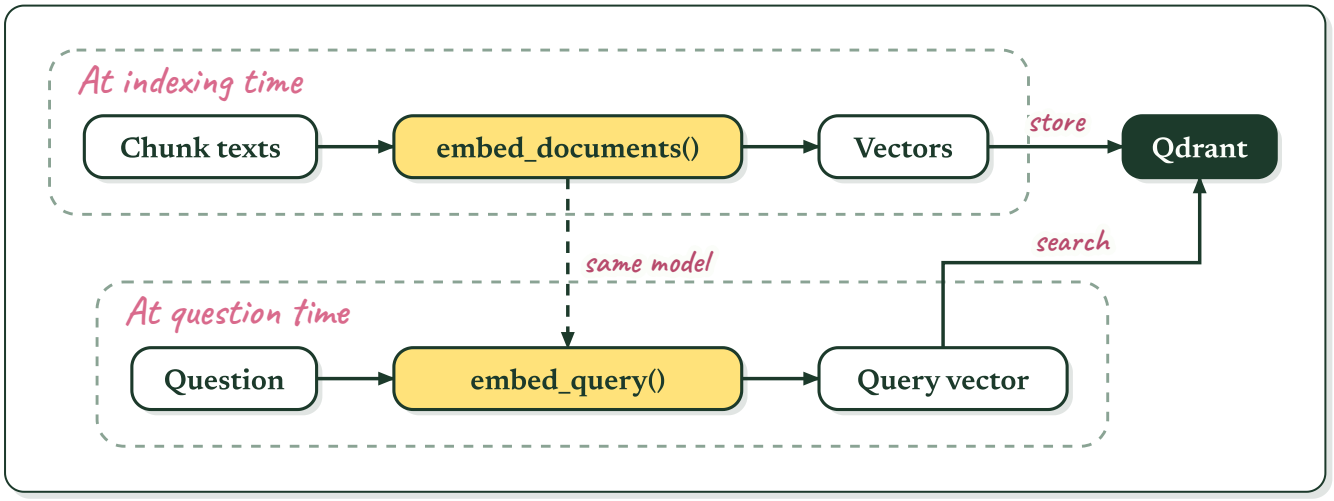
---

part four

Vectors and the vector store

Embeddings in LangChain

LangChain gives every embedding provider the same two methods. One embeds many texts at once for indexing. The other embeds a single question for searching. Because both come from the same object, the query and the documents always live in the same vector space.



```python
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

texts = [
    "Full-time employees receive 24 days of paid annual leave.",
    "Client meals are reimbursed up to the per-person limit with a receipt."
    "Restart the payments service from the runbook, never from the console."
]

vectors = embeddings.embed_documents(texts)
print("Number of vectors:", len(vectors))        # 3, one per text
print("Vector dimension:", len(vectors[0]))      # 1536 for this model
print("First 5 values:", vectors[0][:5])

query_vector = embeddings.embed_query("How many vacation days do I get?")
print("Query vector dimension:", len(query_vector))
```

Copy

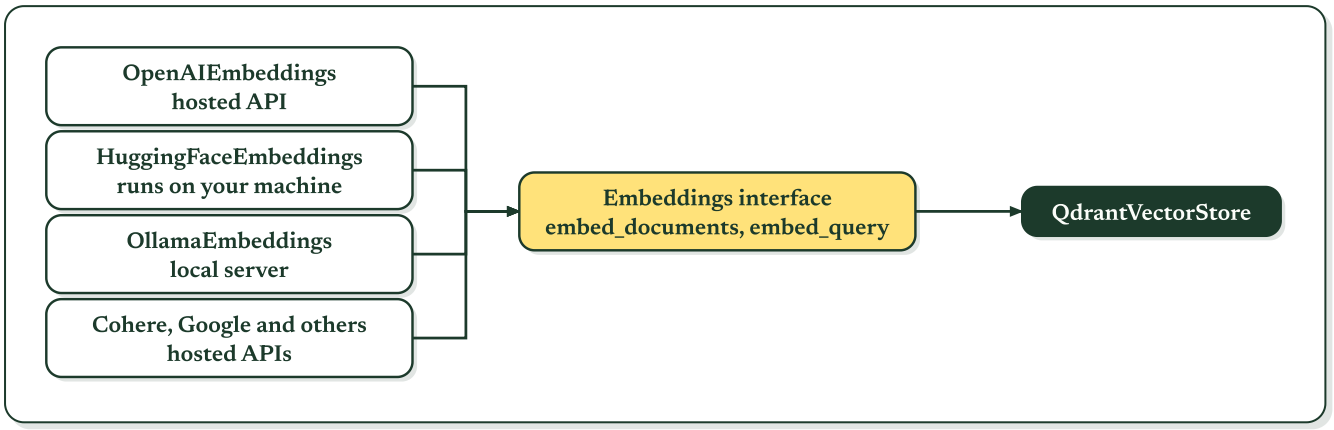
Notice the question says "vacation days" while the handbook says "paid annual leave". Embeddings match *meaning*, not exact words, and that is what makes semantic search feel smart.

> ***watch out***
> Do not hard-code the dimension. Read it from a real vector, as above, whenever you create a collection. If you later switch embedding models the dimension changes, and Qdrant will reject mismatching vectors.

> ***Go deeper*** Embedding integrations OpenAI embeddings guide

Embedding providers compared

Every embedding class exposes `embed_documents()` and `embed_query()`, so the rest of your pipeline neither knows nor cares which one you picked. That makes the choice a business decision about cost, privacy and quality.



Provider	Where it runs	Pros	Cons	Choose it when
`OpenAIEmbeddings`	OpenAI's servers	Strong quality, zero setup, scales instantly	Per-token cost. Your text leaves your network.	You want a reliable baseline fast
`HuggingFaceEmbeddings`	Your CPU or GPU	Free per call, private, hundreds of models	You manage hardware. Larger models need a GPU.	Data cannot leave the machine, or volume makes API cost large
`OllamaEmbeddings`	A local Ollama server	Easy local setup, private	Smaller model choice, you run the server	Offline demos and private prototypes

Provider	Where it runs	Pros	Cons	Choose it when
Cohere, Google and others	Vendor servers	Competitive quality, multilingual options	Another vendor and key to manage	You already use that vendor, or need a specific language strength

```
# Option 1: hosted API (1536 dimensions)
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Option 2: Using OpenRouter
embeddings = OpenAIEmbeddings(
    model="openai/text-embedding-3-small",
    openai_api_base="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
)

# Option 3: a free local model (384 dimensions), no API key
# pip install langchain-huggingface sentence-transformers
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-Min
```

Copy

watch out

Swapping the embedding model means **re-embedding and re-indexing everything** into a new collection. Dimensions differ (1536 against 384), and even equal-sized vectors from different models are not comparable.

Qdrant through LangChain

This is the key step from the raw SDK to the abstraction. LangChain does **not** replace Qdrant. It adds a standard vector-store interface on top of it, and `qdrant-client` still does the real work underneath.



Qdrant stores each chunk as a **point**: the vector, plus a payload holding the chunk text and its metadata. The LangChain wrapper creates the collection, embeds your chunks and uploads everything in one call.

> *Go deeper* Qdrant with LangChain QdrantVectorStore reference

Vector stores compared

We use Qdrant, but LangChain has a vector-store wrapper for dozens of databases, and the code you write against the interface stays almost identical. Knowing the alternatives helps you justify the choice, and change it later if you need to.

Store	What it is	Strengths	Limits	Choose it when
Qdrant	Open-source vector database, self-hosted or cloud	Rich payload filtering, hybrid search, quantization, local mode	You operate a server unless you use the cloud	You need filtering, multi-tenancy and a path to production
Chroma	Lightweight embedded database	Very quick to start, simple API	Fewer production and scaling features	Notebooks and small prototypes
FAISS	A similarity-search library,	Extremely fast in-process	No server, weak	Research and fully offline

Store	What it is	Strengths	Limits	Choose it when
	not a database	search	metadata filtering, you manage persistence	experiments
pgvector	Vector search inside PostgreSQL	One database for data and vectors, SQL filters	Less specialised than a vector-first database at large scale	You already run Postgres and want fewer moving parts
Pinecone	Fully managed vector service	No servers to run, scales automatically	Proprietary, ongoing cost	You want zero infrastructure work

Build the vector store

Start with in-memory mode. It needs no server, so the whole notebook runs anywhere. The stored data disappears when the Python process ends, which is perfect for experiments.

```
from langchain_qdrant import QdrantVectorStore

vector_store = QdrantVectorStore.from_documents(
    documents=chunks,
    embedding=embeddings,
    ids=ids,                          # the stable IDs from the previous ste
    location=":memory:",
    collection_name="helpdesk_kb",
)

print("Vector store ready")
```

Copy

In one call this ran the pipeline described earlier:

Chunks → Embed each chunk → Create the collection → Store vectors + payload

> *note*
> This step calls the embedding API once per batch of chunks, so it costs a little money and takes seconds. That is why you build the store once, then persist it (next section) instead of rebuilding on every run.

Persisting data and running a server

In-memory mode is a great teaching tool but a poor home for real data. Qdrant offers a ladder of deployment modes, and the LangChain code barely changes between them.

Deployment :memory: → Local folder → Docker server → Managed cloud

Mode	How	Use it for
In memory	`location=":memory:"`	Experiments and tests
Local folder	`path="./qdrant_data"`	Persistence on one machine
Server	`url="http://localhost:6333"`	Real applications and shared development
Cloud	`url=...` and `api_key=...`	Production without running servers

Run Qdrant in Docker

```
docker run -p 6333:6333 -v "$(pwd)/qdrant_storage:/qdrant/storage" qdrant/qdr
```

Copy

Open `http://localhost:6333/dashboard` to browse your collections in the Qdrant web UI. Then point LangChain at the server. Only the connection arguments change:

```
vector_store = QdrantVectorStore.from_documents(
    documents=chunks,
    embedding=embeddings,
    ids=ids,
    url="http://localhost:6333",
    collection_name="helpdesk_kb",
)
```

Copy

Full control over the collection

If you want to decide the collection settings yourself, create it with `qdrant-client` and hand the client to LangChain. This is also where you set the dimension from a real vector and choose the distance metric.

```
from qdrant_client import QdrantClient, models

client = QdrantClient(url="http://localhost:6333")
dimension = len(embeddings.embed_query("dimension probe"))

if not client.collection_exists("helpdesk_kb"):
    client.create_collection(
        collection_name="helpdesk_kb",
        vectors_config=models.VectorParams(size=dimension, distance=models.D
    )

vector_store = QdrantVectorStore(client=client, collection_name="helpdesk_kb
```

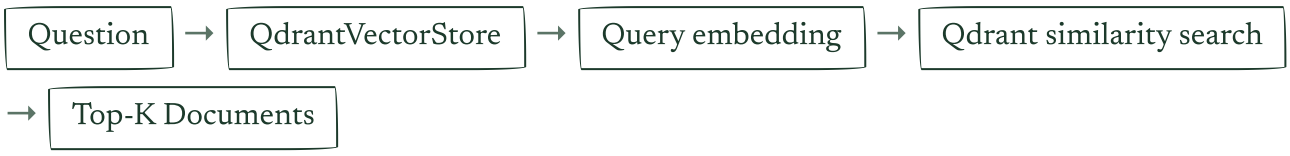
Copy

> *Go deeper* Qdrant quickstart Qdrant + LangChain: existing collections

---

Similarity search

Before building the full chain, test the vector store by itself. Search is the heart of RAG, and it is easy to check by eye.




```
question = "How many vacation days do I get?"

results = vector_store.similarity_search(question, k=3)

for rank, doc in enumerate(results, start=1):
    print(f"\n--- Result {rank} ---")
    print(doc.metadata["filename"], doc.metadata.get("page"))
    print(doc.page_content[:250])
```

Copy

Add scores when you want to see *how* close each match is:

```
for doc, score in vector_store.similarity_search_with_score(question, k=3):
    print(round(score, 3), doc.metadata["filename"], doc.metadata.get("page"))
```

Copy

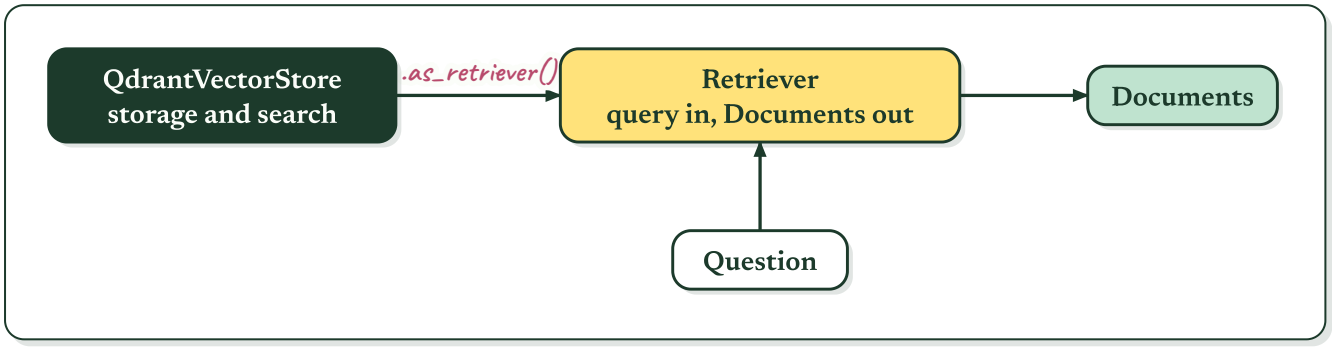
```
0.641 employee_handbook.pdf 1
0.512 employee_handbook.pdf 2
0.287 onboarding.md None
```

The first two hits are the leave policy. The third is an onboarding page that only mentions time off in passing. Scores here are illustrative. Your numbers will differ.

> ***watch out***
> The meaning of a score depends on the distance metric. With cosine in Qdrant, higher means more similar. Other databases return *distances*, where lower is better. Never assume the direction without checking.

From vector store to retriever

A retriever is a thinner interface with a single job: *query in, Documents out*. The vector store knows about storage and search. The retriever hides all of that behind one method, which is why the rest of your chain does not change if you later swap in a smarter retrieval strategy.




```
retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4},
)

docs = retriever.invoke("What is the process for reimbursing a client dinner

for doc in docs:
    print(doc.metadata["filename"], doc.metadata.get("page"))
```

Copy

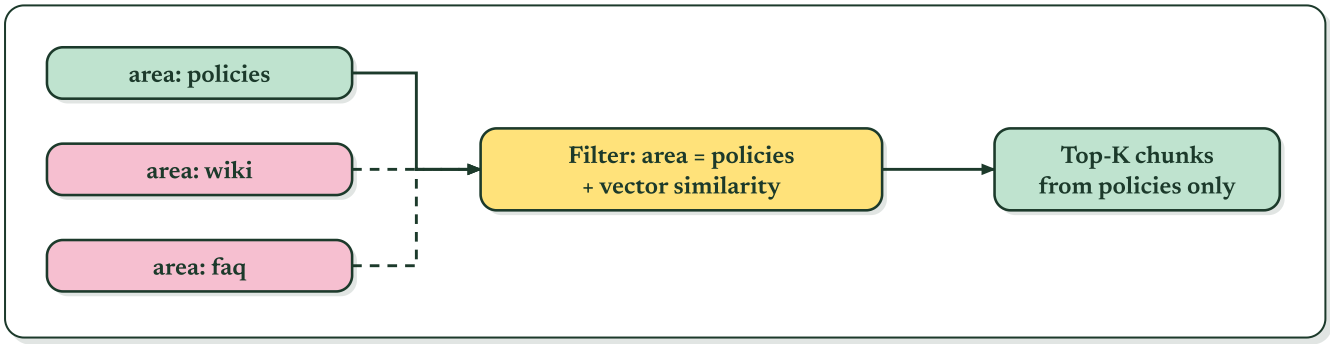
Setting	What it does
`search_type="similarity"`	Plain nearest-neighbour search. The default and a good baseline.
`search_type="similarity_score_threshold"`	Returns only results above a score. Set it with `search_kwargs={"score_threshold": 0.5}`.
`search_type="mmr"`	Balances relevance and variety, so you do not get five near-copies of one paragraph.

Setting	What it does
`search_kwargs={"k": 4}`	How many Documents to return.

> *Go deeper* LangChain retrieval

Filtering with metadata

Similarity alone is not enough in a real product. Maybe a user wants answers from the *policies only*, not the wiki. Or, in a multi-customer system, one tenant must never see another's documents. Qdrant lets you combine similarity with a filter on the metadata you stored. This is where the `area` and `tenant_id` fields you added earlier pay off.




```
from qdrant_client import models

only_policies = models.Filter(
    must=[
        models.FieldCondition(
            key="metadata.area",                    # LangChain stores meta
            match=models.MatchValue(value="policies"),
        )
    ]
)

results = vector_store.similarity_search(
    "Can I expense a client dinner?",
    k=4,
```

```
    filter=only_policies,
)

for doc in results:
    print(doc.metadata["filename"], doc.metadata.get("page"))
```

You can also bake the filter into a retriever so the rest of the chain never has to think about it:

```
retriever = vector_store.as_retriever(search_kwargs={"k": 4, "filter": only_p
```

Copy

> *tip*
> Index the fields you filter on, for example `metadata.tenant_id`, with `client.create_payload_index(...)` so filtered search stays fast on large collections.

> *watch out*
> A filter is your **security boundary**. Enforce it in code, from the signed-in user's identity.
>
> Never ask the LLM to "only use documents from this tenant".

> *Go deeper* Qdrant filtering

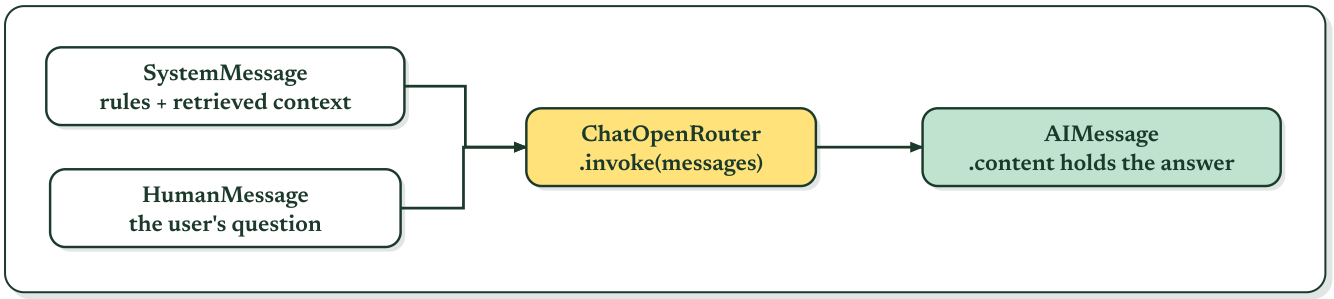
---

part five

Retrieve, prompt and answer

The chat model

Retrieval is done. Now the generation side. In LangChain a chat model takes a list of **messages** and returns one message back. Messages have roles: a system message sets the rules, a human message carries the question, and the model answers with an AI message.



We use **OpenRouter**, a single API that reaches many models. LangChain has a dedicated integration, so prefer it over pointing `ChatOpenAI` at a custom URL.

```
from langchain_openrouter import ChatOpenRouter

llm = ChatOpenRouter(
    model="openai/gpt-4.1-mini",      # any model id from openrouter.ai/models
    temperature=0,                    # 0 keeps answers focused and repeatable
)

response = llm.invoke([
    ("system", "You are a concise assistant."),
    ("human", "Explain RAG in one sentence."),
])

print(response.content)
```

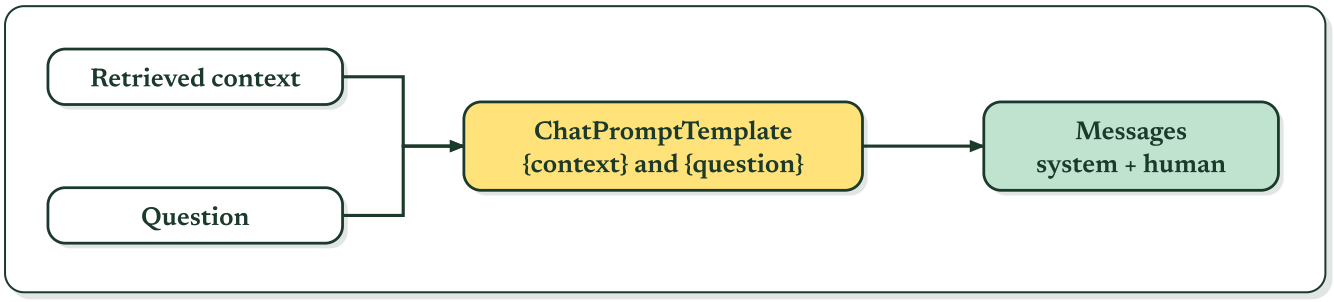
tip

Because the model is behind an interface, switching is a one-line change. `ChatGroq(model="llama-3.3-70b-versatile")` from `langchain_groq`, or another OpenRouter model id, drops into the same chain without touching retrieval.

Go deeper ChatOpenRouter reference OpenRouter docs

The RAG prompt

The model must receive two things: the retrieved **context** and the user's **question**. A `ChatPromptTemplate` is a reusable message list with named blanks to fill in.




```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        """You are Zomato's internal help-desk assistant.
Answer using only the context below.

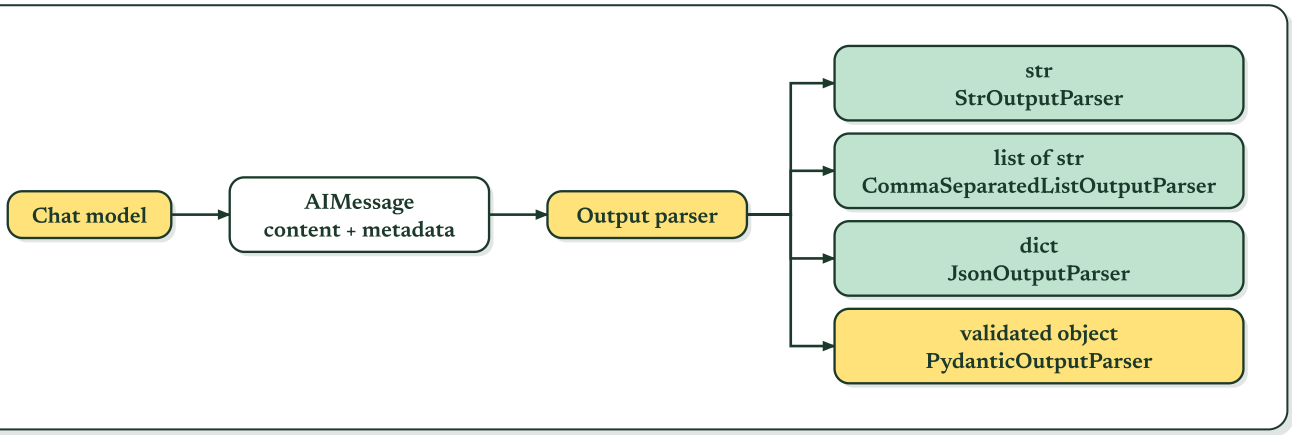
Rules:
- If the answer is not in the context, say you don't know.
- Cite the sources you used as [1], [2], matching the numbers in the context
- Never invent policies, amounts or procedures.

Context:
{context}""",
    ),
    ("human", "{question}"),
])
```

Those rules do real work. "Use only the context" and "say you don't know" are what turn an LLM from a confident guesser into a grounded assistant. Treat the prompt as code: version it, review it and test it.

Output parsers

A chat model does not return a plain string. It returns a message object, an `AIMessage`, which carries the text in `.content` together with metadata such as token usage. That is fine for a quick test, but real code usually wants something simpler or more structured: a clean string to show a user, a list of keywords, or a dictionary to branch on. An **output parser** is the last link in a chain that converts the model's message into exactly that.



Parser	Returns	Use it for
`StrOutputParser`	A plain string	The final answer in a RAG chain. This is the one you will use most.
`CommaSeparatedListOutputParser`	A Python list of strings	Keywords, tags, alternative search phrases
`JsonOutputParser`	A dictionary	Quick structured replies where you do not need strict checking
`PydanticOutputParser`	A validated Python object with typed fields	Anything your code depends on: routing, extraction, query planning. The most important one for real applications.

StrOutputParser: just the text

Without a parser you must remember to dig the text out of the message every time. With one, the chain ends in a string.

```
from langchain_core.output_parsers import StrOutputParser

message = llm.invoke("Name one benefit of RAG in five words.")
print(type(message).__name__)         # AIMessage
print(message.content)                # the text, plus metadata we did not a

chain = llm | StrOutputParser()
answer = chain.invoke("Name one benefit of RAG in five words.")
print(type(answer).__name__)          # str
print(answer)
```

Copy

CommaSeparatedListOutputParser: a real Python list

Many parsers can also tell the model *how* to format its reply. You paste that into the prompt with `get_format_instructions()`, so the prompt and the parser always agree. Here the model suggests search keywords for a question, and we get back a list we can loop over.

```
from langchain_core.output_parsers import CommaSeparatedListOutputPars
from langchain_core.prompts import ChatPromptTemplate

list_parser = CommaSeparatedListOutputParser()

keyword_prompt = ChatPromptTemplate.from_template(
    "Give 3 short search keywords for this question.\n{format_instructions}\n
).partial(format_instructions=list_parser.get_format_instructions())

keyword_chain = keyword_prompt | llm | list_parser

print(keyword_chain.invoke("How many days of paid leave do I get?"))
```

Copy

```
['paid leave', 'annual leave days', 'leave policy']
```

JsonOutputParser: a dictionary you can branch on

When your code needs to make a decision, ask for JSON. A common use in RAG is **routing**: let the model pick which part of the knowledge base a question belongs to, then search only there. This is the quick version. The next parser makes it safer.

```
from langchain_core.output_parsers import JsonOutputParser

json_parser = JsonOutputParser()

route_prompt = ChatPromptTemplate.from_template(
    "Decide which area of the Zomato knowledge base best answers the questio
    'Reply with JSON containing "area" (policies, wiki, faq or catalog) and a
    "{format_instructions}\nQuestion: {question}"
).partial(format_instructions=json_parser.get_format_instructions())

route_chain = route_prompt | llm | json_parser

decision = route_chain.invoke("How do I restart the payments service?")
print(decision["area"])          # wiki
```

Copy

```
{'area': 'wiki', 'reason': 'This is an operational procedure from an enginee
```

That `area` value is exactly what you would feed into the metadata filter from the filtering section, so the search only looks inside the right documents. But a dictionary trusts the model completely, which is where the last parser comes in.

PydanticOutputParser: a typed, validated object

A dictionary is flexible, and that is its weakness. Nothing stops the model from returning `"Area"` instead of `"area"`, a number where you expected text, or a value you never allowed. `PydanticOutputParser` closes that gap. You describe the result as a **Pydantic model**, with typed fields and rules, and the parser both tells the model the exact format to use and **validates** the reply against it. What you get back is a real Python object, not a loose dictionary.

Pydantic model → Format instructions in the prompt → Chat model reply →




```
from typing import Literal

from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser


class QueryPlan(BaseModel):
    area: Literal["policies", "wiki", "faq", "catalog"] = Field(description="
    topic: str = Field(description="The topic in 2 or 3 words")
    needs_exact_numbers: bool = Field(description="True if amounts or dates a


plan_parser = PydanticOutputParser(pydantic_object=QueryPlan)

plan_prompt = ChatPromptTemplate.from_template(
    "Analyse the question.\n{format_instructions}\nQuestion: {question}"
).partial(format_instructions=plan_parser.get_format_instructions())

plan_chain = plan_prompt | llm | plan_parser

plan = plan_chain.invoke("What is the limit for client dinners?")

print(type(plan).__name__)          # QueryPlan, a real object
print(plan.area)                    # policies
print(plan.topic)                   # client dinner limit
print(plan.needs_exact_numbers)     # True
```

Copy

Look at what each part of the model does:

- **Types and choices.** `Literal[...]` limits `area` to four allowed values. If the model invents a fifth, validation fails instead of passing a bad value into your code.
- **Field descriptions are instructions.** The `description` texts are included in the format instructions sent to the model, so well-written descriptions give better results.
- **A usable object.** You write `plan.area` with editor autocomplete, not `plan["area"]` and a hope that the key exists.

- **Reusable.** The same model can describe an API response, a database record or a function's output elsewhere in your project.

The routing example from the JSON section becomes safer this way. The validated `plan.area` can go straight into the metadata filter, because you know it is one of the four real areas.

> *tip*
> Many chat models also offer `llm.with_structured_output(QueryPlan)`, which asks the model for the schema through its native tool-calling feature. It is often more reliable than prompt instructions, but it needs a model that supports tool calling. The parser approach works with any chat model, and understanding it explains what `with_structured_output` is doing for you.

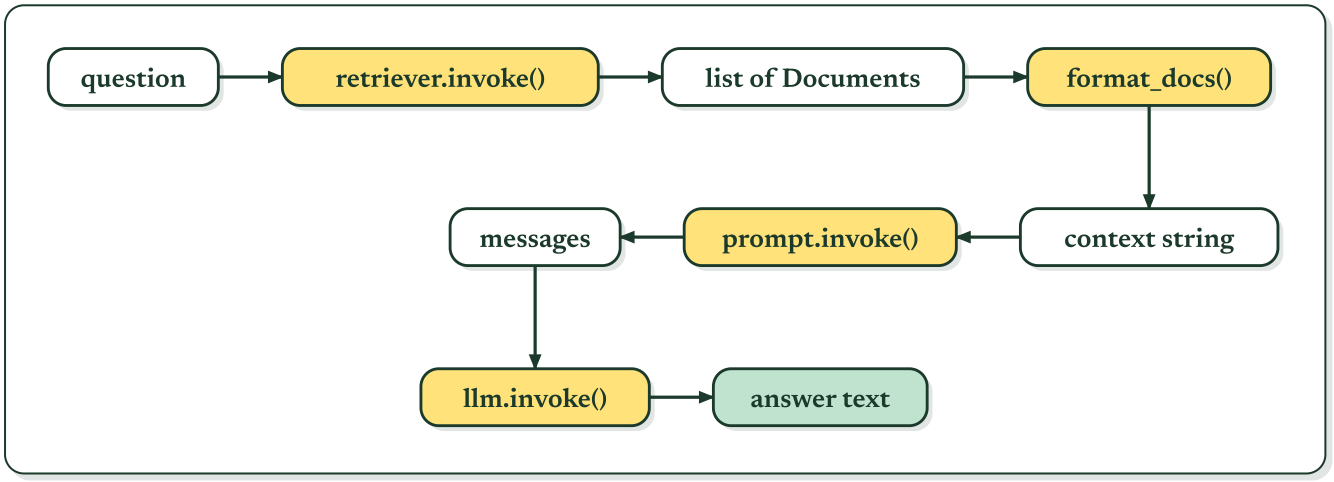
	JsonOutputParser	**PydanticOutputParser**
Returns	A plain dictionary	A typed object
Checks field names and types	No	Yes
Limits values to a fixed set	No	Yes, with `Literal` or enums
On a bad reply	You may find out later, when a key is missing	Raises an error immediately, so you can retry
Best for	Quick experiments	Anything your code depends on

> *watch out*
> A model can still return malformed output, such as JSON with a missing quote. Keep the temperature at 0, state the format clearly in the prompt, and wrap parsing in a `try` block, catching `OutputParserException`, so you can retry when a wrong format would break your application.

> *Go deeper* Output parsers reference

The explicit RAG pipeline

Before using any clever composition syntax, write the pipeline by hand, one visible step at a time. This is the best version to learn from, because you can print and inspect everything in between.



One detail first. Different loaders produce different metadata. A PDF Document has a `page`, a Markdown chunk has headings, and a CSV row has a `row` number. So write **one small helper** that turns any Document into a human-readable citation, and use it everywhere.

```
def cite(doc) -> str:
    m = doc.metadata
    if "page" in m:
        return f"{m['filename']}, page {m['page'] + 1}"          # PDF: pag
    section = m.get("h3") or m.get("h2") or m.get("h1")
    if section:
        return f"{m['filename']}, section '{section}'"           # Markdown
    return m["filename"]                                         # CSV and


def format_docs(docs) -> str:
```

```
    """Number each chunk and label it with its origin so the LLM can cite it
    return "\n\n---\n\n".join(
        f"[{i}] ({cite(d)})\n{d.page_content}" for i, d in enumerate(docs, s
    )
```

explicit_rag.py

```
question = "How many days of paid leave do full-time employees get?"

docs = retriever.invoke(question)                                    # 1. ret
context = format_docs(docs)                                          # 2. for
messages = prompt.invoke({"context": context, "question": question})  # 3. b
response = llm.invoke(messages)                                      # 4. gen

print(response.content)
```

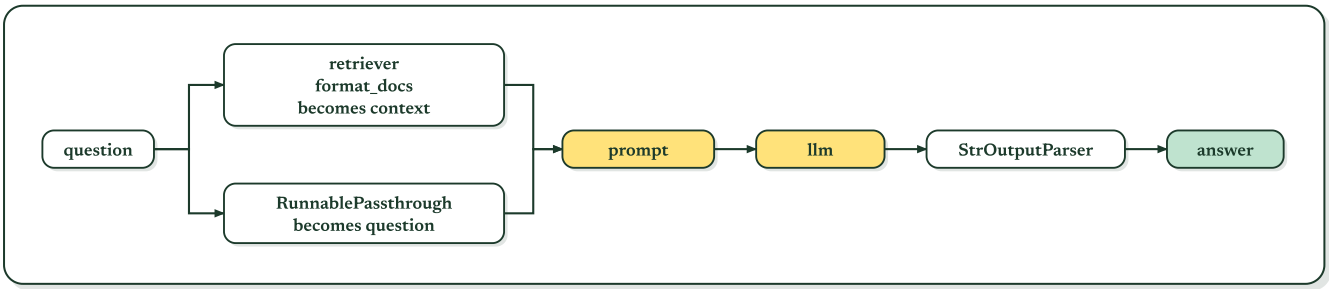
Copy

```
Full-time employees receive 24 days of paid annual leave per calendar year,
5 unused days can be carried over into the next year [1].
```

Every arrow of your pipeline is now a line of code. When something goes wrong, you know exactly which line to print.

The same pipeline as an LCEL chain

Once the explicit version is clear, LangChain lets you express it as a **chain**. Every component is a *Runnable*, and the pipe operator `|` connects them so the output of one becomes the input of the next. The first step is a small dictionary that builds the two prompt inputs in parallel.



```python
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

rag_chain = (
    {
        "context": retriever | format_docs,    # question -> Documents -> con
        "question": RunnablePassthrough(),     # question -> question, unchan
    }
    | prompt
    | llm
    | StrOutputParser()                        # AIMessage -> plain string
)

print(rag_chain.invoke("Which plan includes single sign-on?"))
```

Read it top to bottom as a data flow. The dictionary runs both branches on the same question. The prompt fills its two blanks, the model answers, and the parser strips the message wrapper. Because everything is a Runnable, you get extra methods for free:

```python
# stream tokens as they are generated
for token in rag_chain.stream("How do I set up the company VPN?"):
    print(token, end="", flush=True)

# answer several questions at once
answers = rag_chain.batch([
    "How many days of paid leave do I get?",
    "Can I expense a client dinner?",
])
```

> ***tip***
> Do not memorise the syntax. Memorise the flow: **question, then retrieve and format, then prompt, then model, then parse.** The code is just that sentence written down.

Go deeper LCEL concepts Runnable reference

Debugging a RAG pipeline

When an answer is wrong, do not tweak the prompt at random. Walk the pipeline in order and look at the real data at each stage:

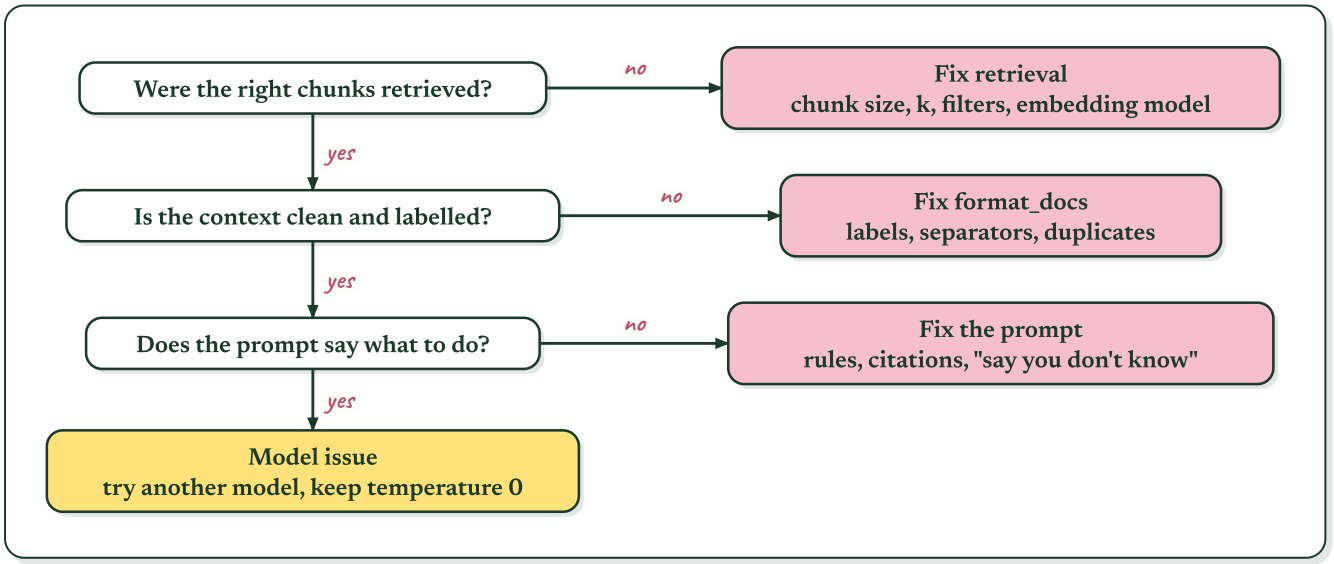
```
question = "How do I restart the payments service?"

docs = retriever.invoke(question)
print("RETRIEVED")
for i, d in enumerate(docs, start=1):
    print(f"[{i}] {cite(d)}: {d.page_content[:120]!r}")

context = format_docs(docs)
print("\nCONTEXT LENGTH:", len(context), "characters")

messages = prompt.invoke({"context": context, "question": question})
print("\nPROMPT MESSAGES")
for m in messages.to_messages():
    print(f"- {m.type}: {m.content[:100]!r}")

print("\nANSWER")
print(llm.invoke(messages).content)
```



The order is deliberate. Retrieval problems are by far the most common cause, so check them first. A model can only answer from what it is given.

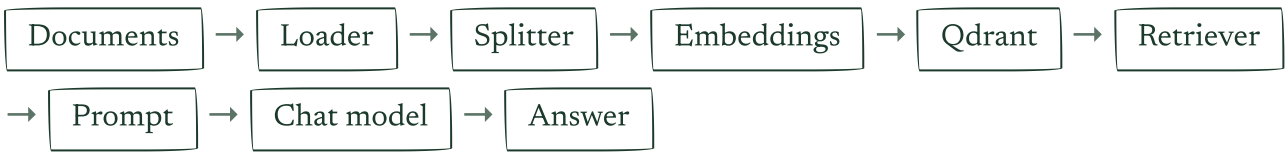
---

part six

Putting it together

The one mental model to keep

Do not think *LangChain equals RAG*. Think of RAG as a pipeline in which each stage is a swappable component, and each component has a clear owner:



LangChain
The orchestration layer. Loaders, splitters, retrievers, prompts and chains.

Qdrant
The retrieval store. Vectors, payload, filtering and fast search.

OpenAI
The embedding provider. Turns text into comparable vectors.

OpenRouter
The generation gateway. Writes the final grounded answer.

The biggest lesson of all: **most of the quality in a RAG system comes from choosing the right loader and the right splitter for each kind of document**, not from the chain at the end. Parse well, chunk along the document's own structure, and the rest is plumbing.

Checklist

Tick these off as you go. Your progress is saved in this browser.

Loading and chunking

- [ ] Explain what a LangChain Document contains
- [ ] Choose between PyPDF, PyMuPDF, PDFPlumber and Unstructured for a PDF
- [ ] Recognise a scanned PDF and use OCR
- [ ] Load CSV, JSON, Word, HTML and web sources
- [ ] Explain recursive, token, header, code, JSON and semantic splitters, and when to use each
- [ ] Route different document types to different splitters
- [ ] Enrich metadata and build stable chunk IDs

Embeddings and Qdrant

- [ ] Create OpenAIEmbeddings and call embed_documents and embed_query
- [ ] Compare embedding providers and vector stores
- [ ] Build a QdrantVectorStore in memory and on a server, and reconnect later
- [ ] Run similarity_search, read the scores, and apply a metadata filter

Generation

- [ ] Call ChatOpenRouter and write a ChatPromptTemplate
- [ ] Build the explicit pipeline, then the LCEL chain
- [ ] Debug a bad answer stage by stage

Self-test

Try to answer each question out loud before opening it.

▶ **A PDF loads as blank pages. What is wrong, and what do you do?**

▶ **Why is RecursiveCharacterTextSplitter a safer default than CharacterTextSplitter?**

▶ **Why split Markdown on headings before splitting by size?**

▶ **When would you size chunks in tokens instead of characters?**

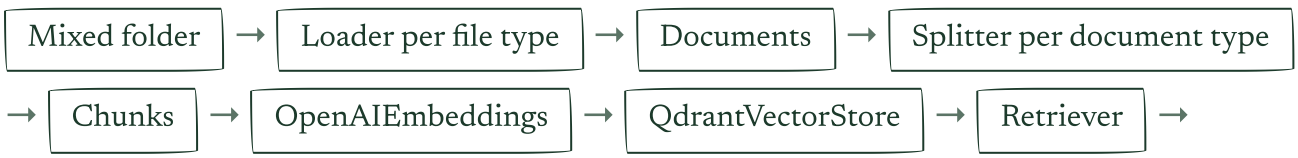
▶ **Why must ingestion use stable IDs?**

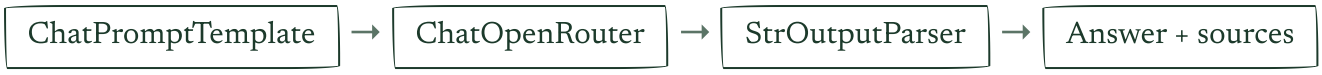
▶ **Why must a tenant filter be enforced in code, not in the prompt?**

▶ **An answer is wrong. What do you check first?**

Final challenge

Explain the complete pipeline in your own words, naming the LangChain component and the choice you made at each step:







Want to go further? Explore the AI Engineering course from Coding Shuttle.